AIML402: Assignment 3 Report

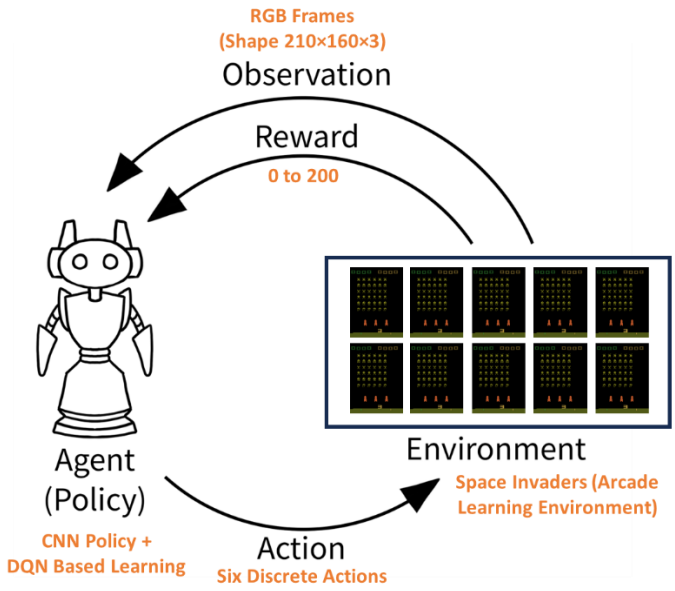
Siddharth Barjatya ( )

October 12th, 2025

This document is a submission for Assignment 3 : Task 2 - A report explaining the model, the DQN implementation, results evaluating the effectiveness of the training and testing on the game of our choice. Most strategies & approaches have been taken from Nature magazine's *Human-level control through deep reinforcement learning* (Mnih, et al., 2015) (Published: 25 February 2015)

Brief Introduction

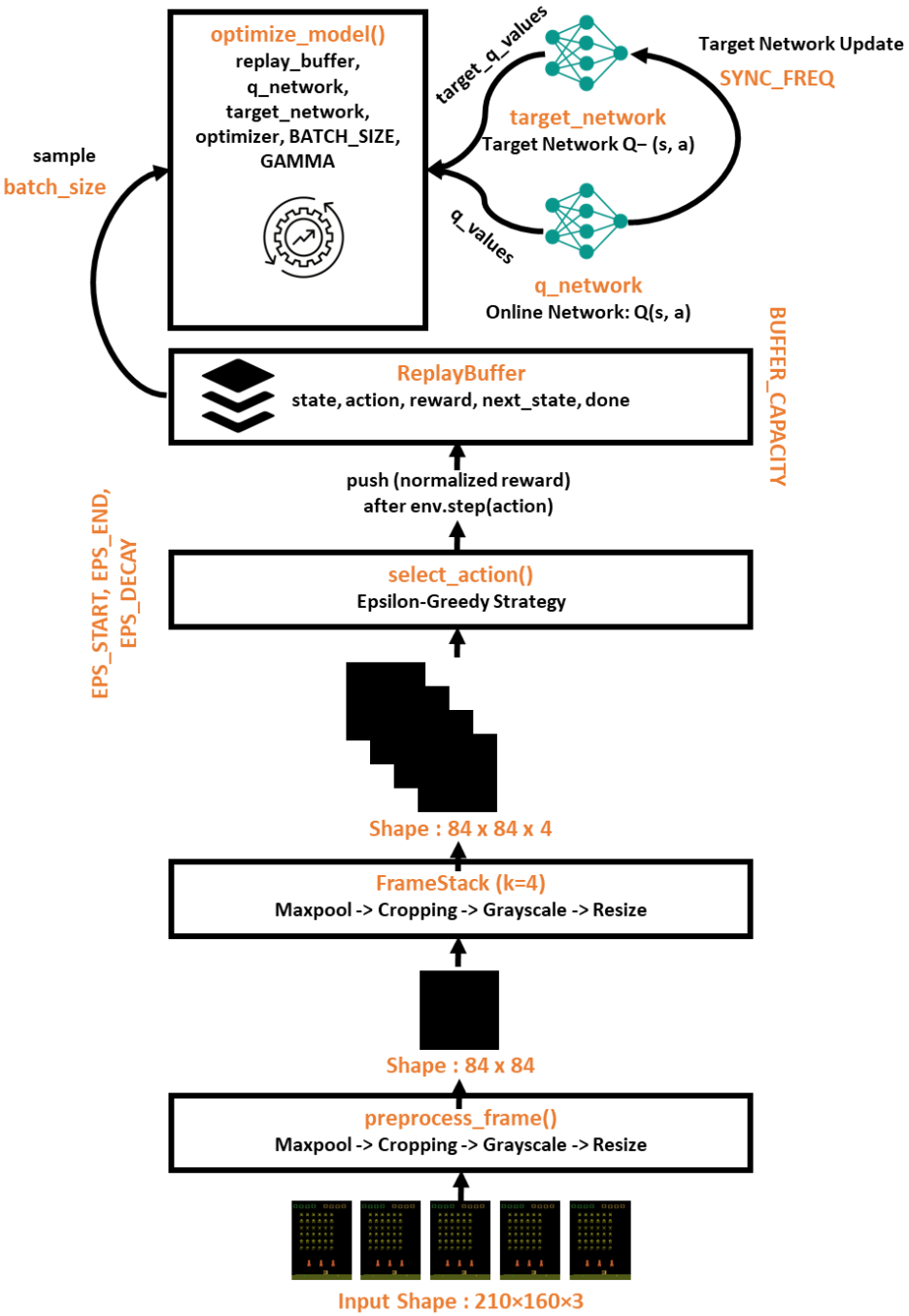
This report presents the implementation and evaluation of a Deep Q-Network (DQN) agent trained to play the Atari game *Space Invaders* using reinforcement learning. This code report specifically addresses the requirements of implementing a DQN-based algorithm capable of learning to play an Atari game through reinforcement learning, using the Arcade Learning Environment (ALE) via the Gymnasium API in Python. *Space Invaders* was chosen as the environment, and the agent was trained using a convolutional neural network to approximate the Q-function. The implementation relies solely on Gymnasium, ALE, and PyTorch, adhering to the project constraints of avoiding any additional libraries. The report details the architecture of the policy network, the reinforcement learning strategy employed, evidence of training through reward progression, and an analysis of the agent's performance.



The primary objective of the game is to control a laser cannon and eliminate waves of alien invaders before they reach the bottom of the screen or deplete the player's three lives. Points are earned for destroying invaders, with higher rewards for targets positioned further back and occasional bonus points for hitting command ships. Protective shields provide temporary cover, but are gradually destroyed as the game progresses. The game ends if the lowest invaders reach the bottom of the screen. The action space consists of six discrete actions (no-op, fire, move left/right, and combined

move-and-fire actions), while the observations are RGB frames of shape 210×160×3. Default difficulty and mode settings were used in this code report.

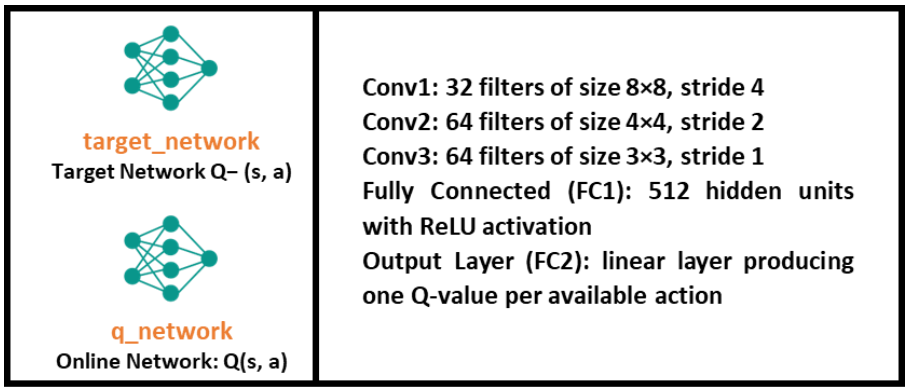
Description of the implementation and reasoning for the choices of the architecture of the neural network model for the policy



Deep Q-Network (DQN) Architecture

The Deep Q-Network (DQN) implemented for this assignment is a convolutional neural network (CNN) designed to approximate the Q-value function for the Atari *Space Invaders* environment. The network receives as input a stack of four consecutive preprocessed grayscale frames, each of dimension 84×84 pixels, forming an input tensor of shape (4, 84, 84). Stacking multiple frames allows

the agent to infer temporal dynamics, such as the velocity and direction of moving objects, from static images. The image preprocessing is taken care by the preprocess_frame () while the stacking is handled by the FrameStack class (described later below). The network architecture consists of three convolutional layers followed by two fully connected layers:



Each convolutional layer is followed by a ReLU activation to introduce non-linearity. After the convolutional feature extraction, the resulting 64 × 7 × 7 = 3136 feature map is flattened and passed through the fully connected layers to generate the estimated Q-values for each possible action. These Q-values represent the expected future cumulative rewards for taking a given action in the current state.

This architecture closely follows the original DQN design proposed by (Mnih, et al., 2015) in *"Human-level control through deep reinforcement learning."* Convolutional layers capture spatial and visual patterns within the game frames, while the fully connected layers map these features to action-value predictions. The choice of three convolutional layers and a 512-unit hidden layer balances expressive capacity with computational efficiency, which is crucial when training on high-dimensional pixel inputs. During the forward pass, raw image inputs are normalized by dividing pixel values by 255.0 to improve numerical stability and accelerate learning. The model outputs Q-values directly, from which the agent's policy selects the action with the highest predicted value (subject to ε-greedy exploration during training).

Network Optimization and Target Network (optimize_model() function)

To train the DQN, a target network and main Q-network are used to stabilize learning. At each training step, a mini-batch of transitions is sampled from the replay buffer. The optimization procedure follows the standard DQN update:

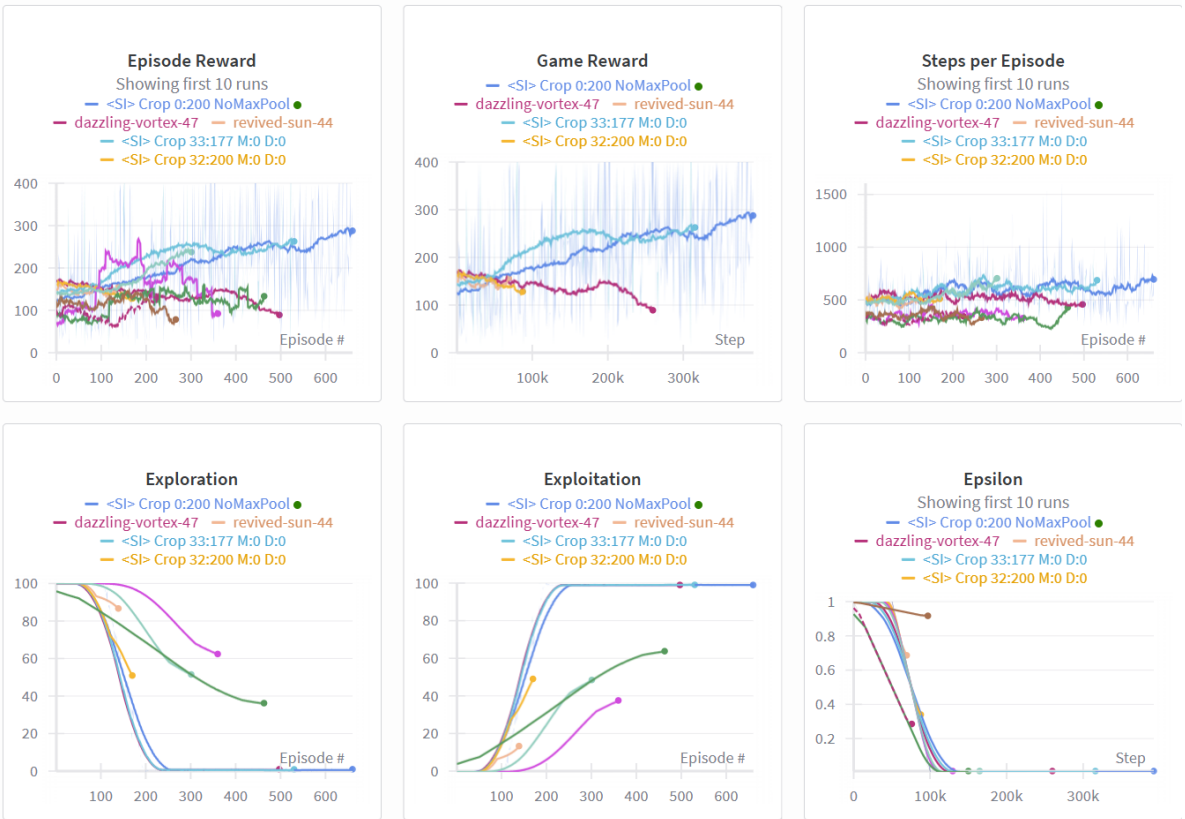
- **Predicted Q-values:** For each state in the batch, the main network predicts Q-values for all actions, and the Q-value corresponding to the action actually taken is selected.
- **Target Q-values:** Using the target network, the maximum Q-value for each next state is computed. The target for the loss is calculated.
- **Loss Computation and Backpropagation:** The mean squared error (MSE) between predicted and target Q-values is minimized using an optimizer (e.g., Adam). Gradients are clipped between -1 and 1 to prevent instability from exploding gradients, a common issue in deep RL.

This procedure is repeated at each step after collecting sufficient experience in the buffer. The target network is updated periodically from the main network to provide a stable reference for target Q-values, which reduces oscillations and divergence during training.

Hyperparameter Tuning

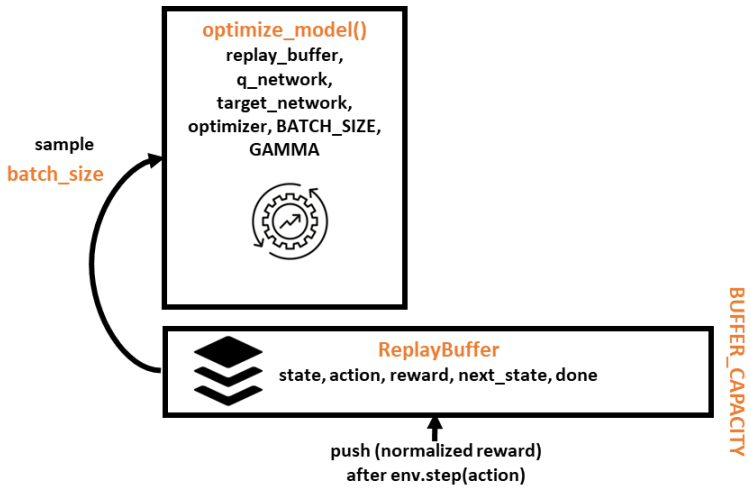
S#	**Batch Size**	**Learning Rate**	**Epsilon Start**	**Epsilon End**	**Observations/Notes**
1	32	1e-4	1.0	0.1	Slow learning, stable but lower reward progression
2	32	1e-4	1.0	0.01	Better exploration early, slower convergence
3	32	2.5e-4	1.0	0.1	Faster learning, slightly unstable at start
4	32	2.5e-4	1.0	0.01	Decent performance: stable reward increase, converges well
5	64	1e-4	1.0	0.1	Smooth learning, slower updates due to smaller gradient steps
6	64	1e-4	1.0	0.01	Slightly faster learning than exp. 5, still unstable
7	64	2.5e-4	1.0	0.1	Fast early learning, some reward spikes
8	64	2.5e-4	1.0	0.01	**Best configuration: stable, smooth reward increase, balanced exploration/exploitation**

- Batch size of 64 provides smoother gradient updates and slightly faster convergence compared to 32. Optimization consumes more resources & time.
- Learning rate of 2.5×10^{-4} allows faster learning while remaining stable. Recommended setting by (Mnih, et al., 2015)
- Lower epsilon end (0.01) ensures the agent eventually exploits learned behaviour thoroughly. Epsilon decay starts after buffer warmup of 50K states.
- Epsilon decay set at 5% of total steps balances early exploration and late-stage exploitation. Buffer fills up 100% exactly when epsilon decays completely.



Explanation of other aspects of the DQN-based RL learning

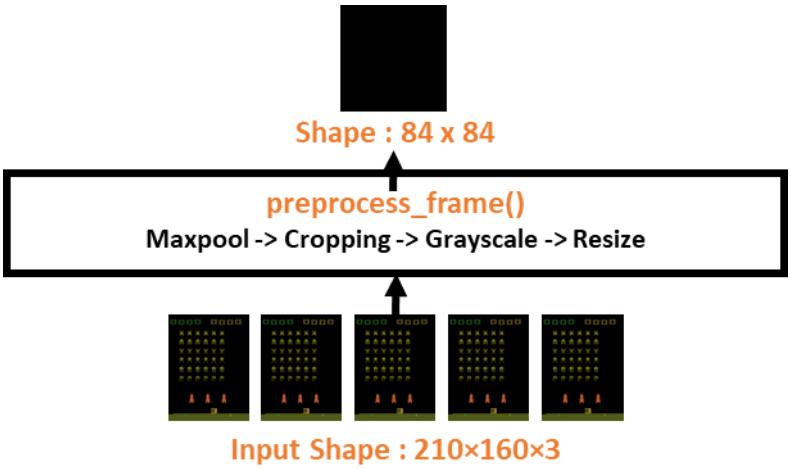
Experience Replay Buffer (ReplayBuffer Class)
To stabilize training and improve sample efficiency, an experience replay buffer was deployed. The buffer stores a finite number of transitions in the form *(state, action, reward, next_state, done)* collected through agent–environment interactions. By sampling mini-batches of past experiences uniformly at random, the DQN breaks the strong temporal correlations between consecutive samples and ensures more stable gradient updates. The replay buffer is implemented using a double-ended queue (deque) with a fixed maximum capacity. When full, older experiences are automatically discarded to make room for new ones. Each training step samples a mini-batch of transitions to compute the temporal-difference (TD) loss and update the network parameters. Before being pushed into the buffer, state observations and next states are converted to NumPy arrays, ensuring efficient serialization. When sampled, all transitions are converted to PyTorch tensors and normalized to the [0, 1] range. The actions and terminal flags are cast to appropriate tensor types (long and float, respectively) for correct computation of the DQN loss function. This mechanism aligns with the methodology described in (Mnih, et al., 2015) and was critical for improving convergence stability, decorrelating data, and enabling the reuse of past experiences multiple times during training.



Frame Preprocessing (preprocess_frame() function)
Each raw observation from the environment, originally an RGB image of resolution 210×160×3, is processed through several stages implemented in the preprocess_frame() function. The output of this function is a single 84×84 grayscale frame that forms the building block for the agent's state representation. These transformations closely follow the preprocessing pipeline described in (Mnih, et al., 2015) :

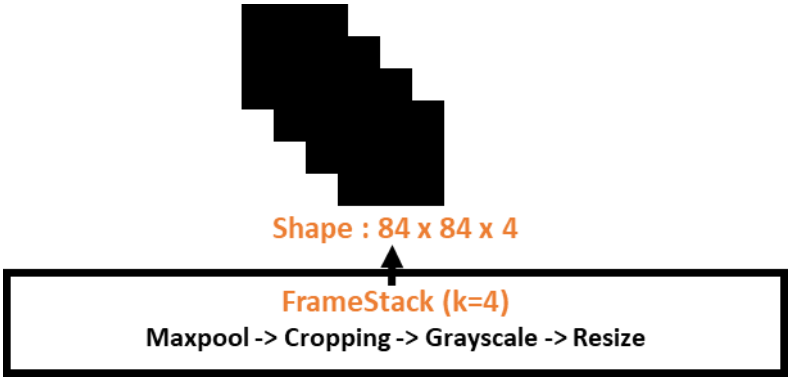
- **Flicker Reduction:** Atari environments often exhibit frame-to-frame flickering due to alternating sprite renderings. This is mitigated by taking the pixel-wise maximum between the current and previous frames. In this implementation, a simplified approach is of env parameter : skipframe =4 was used instead of manually patching the frames together (which was found to be destabilizing learning).
- **Cropping:** The top and bottom portions of the Space Invaders game contains scores and static borders, however the presence of 200 point reward in place of the scoreboard is required to be leant, hence a crop range of 0 to 200 pixels is used (crop_start, crop_end) are configurable depending on the environment.

- **Grayscale Conversion:** The RGB frame is converted to a single grayscale channel using OpenCV's cv2.cvtColor function. This reduces input dimensionality from three color channels to one while preserving essential luminance information that represents object motion and shapes.
- **Resizing:** The cropped grayscale frame is resized to a standardized resolution of 84×84 pixels using area interpolation. This resolution was selected based on the DQN paper (Mnih, et al., 2015) to provide a balance between computational efficiency and visual detail.



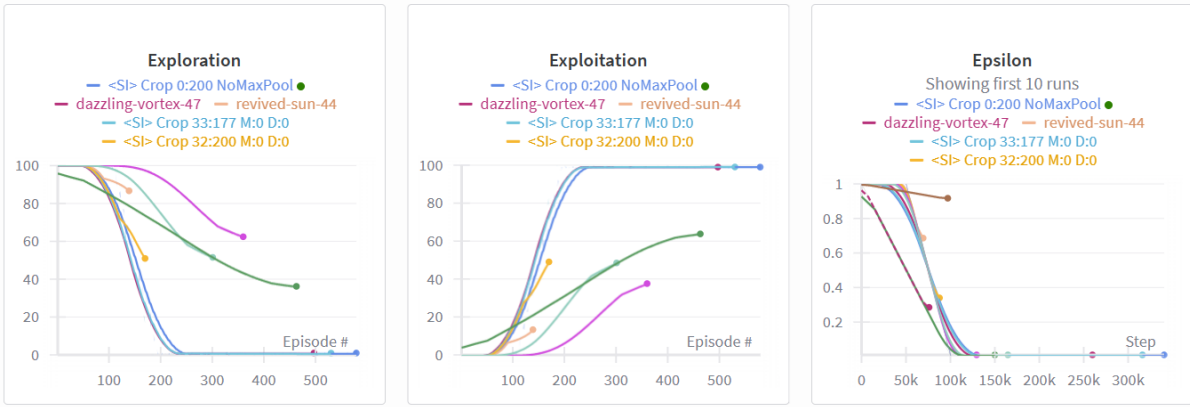
Frame Stacking (FrameStacking Class)
Since a single static image does not convey temporal information such as object velocity or direction, multiple consecutive frames are stacked to form a single state representation. The FrameStack class manages this process. This design enables the agent to perceive motion and temporal dependencies implicitly, allowing it to learn dynamic behaviours such as tracking the trajectory of alien projectiles or anticipating the movement of enemies.

- A deque data structure is used to efficiently maintain a sliding window of the most recent *k* frames (typically *k = 4*).
- Each time a new frame is received, it is pre-processed and appended to the stack, with the oldest frame automatically discarded.
- The stacked frames are then combined along the first (channel) dimension to produce a state tensor of shape (4, 84, 84), which serves as the input to the convolutional layers of the DQN.

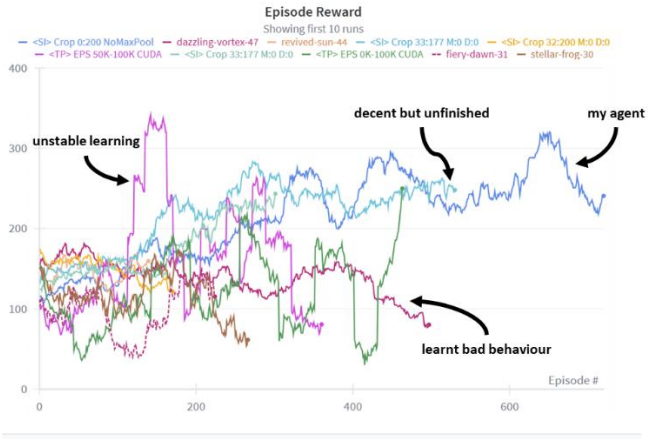


Exploration Strategy: Epsilon-Greedy Policy
The value of ε decays linearly over time to gradually shift the agent's behaviour from exploration to exploitation as training progresses. Initially, ε is set to 1.0, ensuring full exploration during the early

phase of training when the agent's Q-function is untrained. Once the replay buffer accumulates sufficient transitions (after START_LEARNING steps), ε begins to decay linearly until reaching its lower bound ε_end = 0.01. This schedule encourages the agent to explore sufficiently early on while enabling focused exploitation in later stages of learning. The design choice of linear decay is computationally simple, empirically effective, and consistent with the approach proposed in (Mnih, et al., 2015). It ensures a smooth behavioural transition without abrupt policy shifts that could destabilize learning.



Evidence of the model training



Learning Behaviour

Although the initial target was to train the agent for 1 million environment steps, GPU resource constraints limited the training process to approximately **400,000 steps**, corresponding to about **700 episodes** (including multiple lives per episode). Despite the shorter training duration, the DQN agent demonstrated clear evidence of learning and behavioural improvement over time.

The **average episodic reward** increased from around **150** at the start of training to approximately **260** by the end, when smoothed over all 700 episodes. This upward trend indicates that the agent was progressively improving its policy for action selection and survival. The **three highest scores** achieved during training : **880**, **825**, and **805**, further confirm that the agent had learned key strategies that align with reward-maximizing behaviours within the Space Invaders environment, such as better positioning and timing of shots.

Additionally, the **average episode length** increased from roughly **500 steps to 600 steps**, suggesting that the agent was learning to **survive longer** in each episode, thereby accumulating higher rewards. Together, these results reflect that even within a constrained training regime, the DQN framework successfully enabled the agent to extract meaningful policies from the high-dimensional visual input and learn effective gameplay strategies.

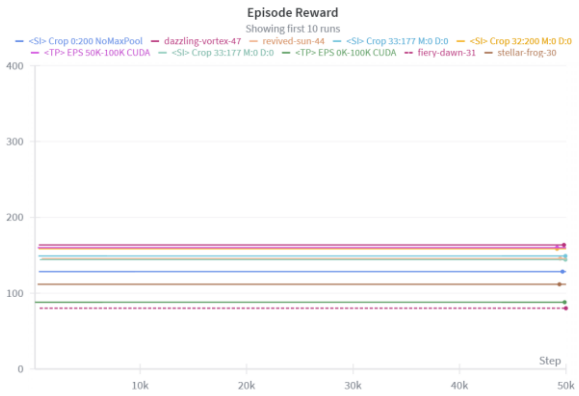


Analysis of the behaviour and the DQN's performance on the game after training

Performance Analysis

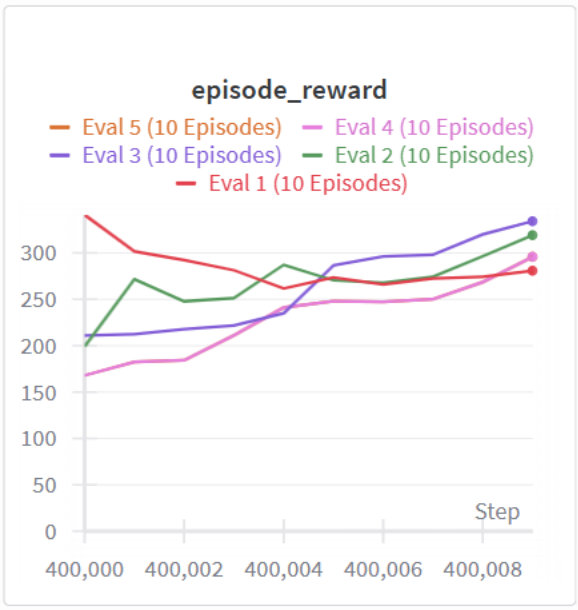
During evaluation, the trained DQN agent demonstrated a substantial improvement in gameplay performance compared to the random policy baseline. The random agent, which selected actions uniformly at random, achieved a mean reward of approximately **130 points** across all evaluation runs. In contrast, the DQN agent consistently achieved an **average reward between 248 and 286 points**, representing roughly a **2x improvement** over the random baseline. Moreover, the **highest rewards** achieved by the DQN agent ranged between **480 and 750 points**, with the best-performing model (Evaluation #3) scoring nearly **six times higher** than the random agent. This clear upward shift in both average and peak rewards indicates that the DQN agent successfully learned to identify and exploit effective strategies within the Space Invaders environment. While variability remains across evaluation runs, suggesting room for further stability and generalization, the results confirm that

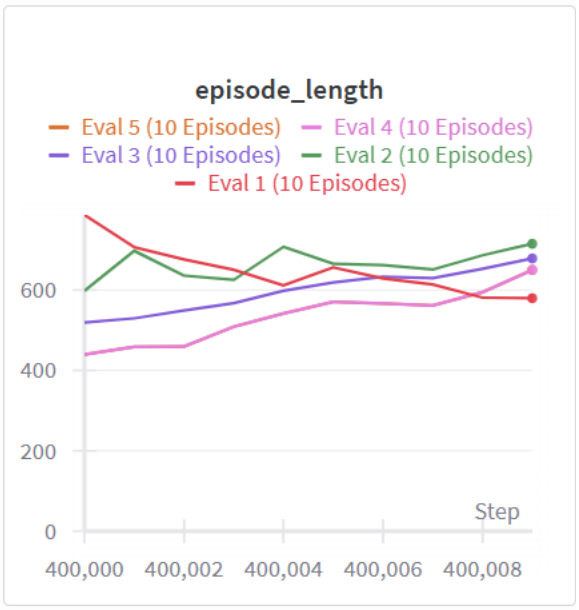
the agent's learning process led to a meaningful increase in policy competence beyond random behaviour.

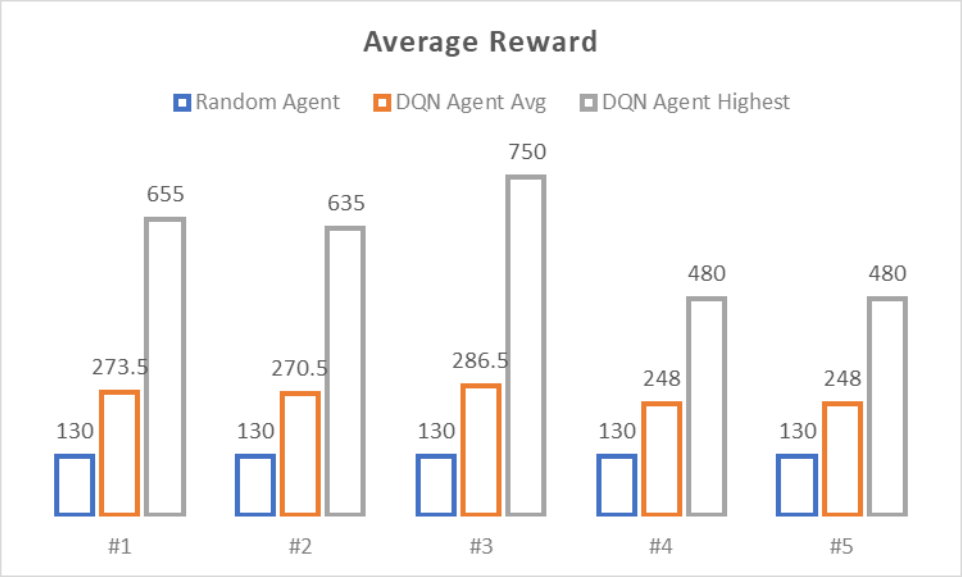


Random Agent

Random Behaviour (First 50K Steps)
Mean Reward : 130 points







Evaluation	Random Agent	DQN Agent Avg	DQN Agent Highest
#1	130	273.5	655
#2	130	270.5	635
#3	130	286.5	750
#4	130	248	480
#5	130	248	480

Behavioural Analysis

Qualitative observations of the trained DQN agent's gameplay behaviour provide additional insight into its learned policy. The agent has developed a basic understanding of offensive movement, often **continuously firing at enemies while maintaining a relatively safe position** behind the in-game defences. This indicates that the agent has learned to associate frequent firing and positional stability with higher short-term rewards. However, the policy still exhibits several limitations. The agent **fails to effectively avoid enemy projectiles**, frequently remaining stationary even when under direct threat, which results in premature episode termination. Moreover, the agent **does not appear to pursue the high-value target** that yields the **200-point reward**, suggesting that it has not yet learned to distinguish between low- and high-reward opportunities. These behaviours imply that while the agent has captured certain low-level patterns through experience, it lacks higher-level strategic understanding and long-term planning, an area that could potentially be improved through extended training, reward shaping, or the introduction of prioritized experience replay.

Conclusion

This assignment focused on developing and evaluating a Deep Q-Network (DQN) agent to play the Space Invaders game using reinforcement learning. The agent was trained using standard techniques including experience replay, frame stacking, and epsilon-greedy exploration, allowing it to learn effective action policies over time. Evaluation results show that the DQN agent significantly outperformed a random policy baseline, achieving average rewards between 248 and 286 points compared to 130 points for the random agent, with top scores reaching 480 to 750 points. Behavioural analysis indicates that the agent learned strong offensive patterns and effective defensive positioning, firing consistently while staying behind cover. However, it still struggles to avoid enemy fire and does not consistently target high-value 200-point enemies, showing that long-term strategic planning is limited. Overall, the agent demonstrates a solid understanding of the game mechanics and short-term reward optimization, with potential for further improvement through additional training and refinement.

Citations

References

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., . . . Kumaran, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 13. doi:10.1038/nature14236